

Module : Transmission de donnees securisee

Rapport de projet

Secure Transmission Protocol Simulator

Conception et realisation d'une application desktop de simulation
d'un mini protocole de transmission securisee sur canal bruite

Présenté Par
Langage principal
Bibliotheques
Livable logiciel

AICH FADI , MAALLEM Mehdi
Python 3.11
PySide6, PyCryptodome, matplotlib, pytest
Application desktop modulaire avec visualisation, logs
et statistiques
23 avril 2026

Date

Table des matières

Introduction	iii
Plan du rapport	iii
1 Contexte et cahier des charges initial	1
1.1 Exigences principales du sujet	1
1.2 Positionnement de la solution realisee	1
2 Architecture generale du projet	2
2.1 Vue d'ensemble	2
2.2 Schema logique de haut niveau	3
2.3 Principes de conception	3
3 Structure du projet dans VS Code	4
3.1 Lecture de l'arborescence	4
4 Description detaillee des modules de code	6
4.1 Package app	6
4.2 Package core.crypto	6
4.3 Package core.coding	6
4.4 Package core.channel	7
4.5 Package core.protocol	7
4.6 Packages core.analytics et core.services	7
4.7 Package ui	7
4.8 Schema des dependances fonctionnelles	8
5 Flux fonctionnel du protocole	9
5.1 Schema sequentiel de la transmission	9
5.2 Interet pedagogique de ce flux	9
6 Analyse de l'interface graphique	11
6.1 Onglet Simulation	11
6.2 Onglet Frame Analysis	12
6.3 Onglet Logs	13
6.4 Onglet Statistics	14
6.5 Onglet Comparison	15
6.6 Onglet Settings	15
7 Validation, qualite logicielle et tests	17

7.1	Strategie de validation	17
7.2	Apports de cette batterie de tests	17
7.3	Lien entre tests et experience utilisateur	17
8	Analyse critique et apports du projet	18
8.1	Apports principaux	18
8.2	Limites	18
8.3	Pistes d'evolution	18
	Conclusion	19
A	Annexe A - Synthese des fonctionnalites	20
B	Annexe B - Fichiers de code les plus importants	21
	Bibliographie	22

Introduction

Ce rapport presente la conception, l'implementation et la validation du projet « Secure Transmission Protocol Simulator ». Le sujet initial demandait un mini protocole de transmission securisee integrant a la fois un mecanisme de chiffrement, une technique de codage ou de detection d'erreurs, une simulation de canal bruite, ainsi qu'une logique de retransmission via ACK/NACK.

Dans la version realisee, l'application va au-dela d'une simple demonstration algorithmique. Elle propose une interface desktop professionnelle, une architecture modulaire separee en couches metier et presentation, des outils de visualisation bit a bit, un journal horodate des evenements, un tableau de bord statistique et un mode d'experimentation comparatif.

Le rapport a donc deux finalites complementaires. La premiere est technique : montrer comment les differentes briques logicielles coopèrent pour assurer la transmission, la validation, la detection des erreurs et la retransmission. La seconde est pedagogique : mettre en evidence, de maniere progressive, les concepts du module a travers une application concrete, manipulable et demonstrable en soutenance.

L'objectif principal est double :

- illustrer de facon claire la relation entre securite, fiabilite et observabilite d'une transmission ;
- fournir un support academique solide pour la demonstration, l'explication du code et l'analyse des choix de conception.

Plan du rapport

Le rapport est organise comme suit :

- le chapitre 1 rappelle le sujet de depart et le cahier des charges ;
- le chapitre 2 decrit l'architecture generale retenue ;
- le chapitre 3 detaille la structure reelle du projet dans VS Code ;
- le chapitre 4 explique les modules Python et les responsabilites de chaque package ;
- le chapitre 5 presente le flux fonctionnel du protocole de transmission ;
- le chapitre 6 analyse l'interface graphique et les tableaux de bord a partir des captures ;
- le chapitre 7 couvre la validation, les tests et la qualite logicielle ;
- le chapitre 8 propose une analyse critique, les apports du projet et ses limites ;
- la conclusion synthetise les resultats obtenus.

1. Contexte et cahier des charges initial

Le sujet d'origine, intitulé « Mini-Protocole de Transmission Securisee », visait la construction d'un simulateur montrant la protection d'un message contre les erreurs et les perturbations d'un canal non fiable. Il fallait relier quatre idees essentielles : le chiffrement, le codage de canal, la transmission bruitée et la retransmission.

1.1 Exigences principales du sujet

Le besoin initial portait sur les points suivants :

- codage de canal par Hamming (7,4) ou CRC ;
- chiffrement du message via AES ou substitution simple ;
- simulation d'un canal avec injection d'erreurs ;
- protocole de base avec envoi, ACK/NACK et retransmission ;
- visualisation du cheminement du message et des corrections appliquees.

1.2 Positionnement de la solution realisee

La solution finale conserve integralement le coeur du cahier des charges, mais l'etend vers une application plus complete :

- interface multi-onglets au lieu d'une simple interface minimale ;
- separation nette entre logique metier, services, analytics et interface graphique ;
- plusieurs modes d'erreur parametrables avec probabilite et seed ;
- journalisation exportable ;
- statistiques sessionnelles et comparaison experimentale entre strategies.

En pratique, cela signifie que le projet n'est plus seulement un exercice de codage, mais une mini-plateforme de simulation. L'utilisateur ne voit pas uniquement un resultat final ; il peut suivre le trajet du message, observer la corruption binaire, verifier le comportement du recepteur et comparer plusieurs strategies dans des conditions identiques.

2. Architecture generale du projet

L'architecture globale suit une chaine logique allant de l'utilisateur vers l'émetteur, le chiffrement, le codage de canal, le canal bruité, le récepteur, la validation, puis le déchiffrement. Cette décomposition facilite la lisibilité du système et la maintenance.

2.1 Vue d'ensemble

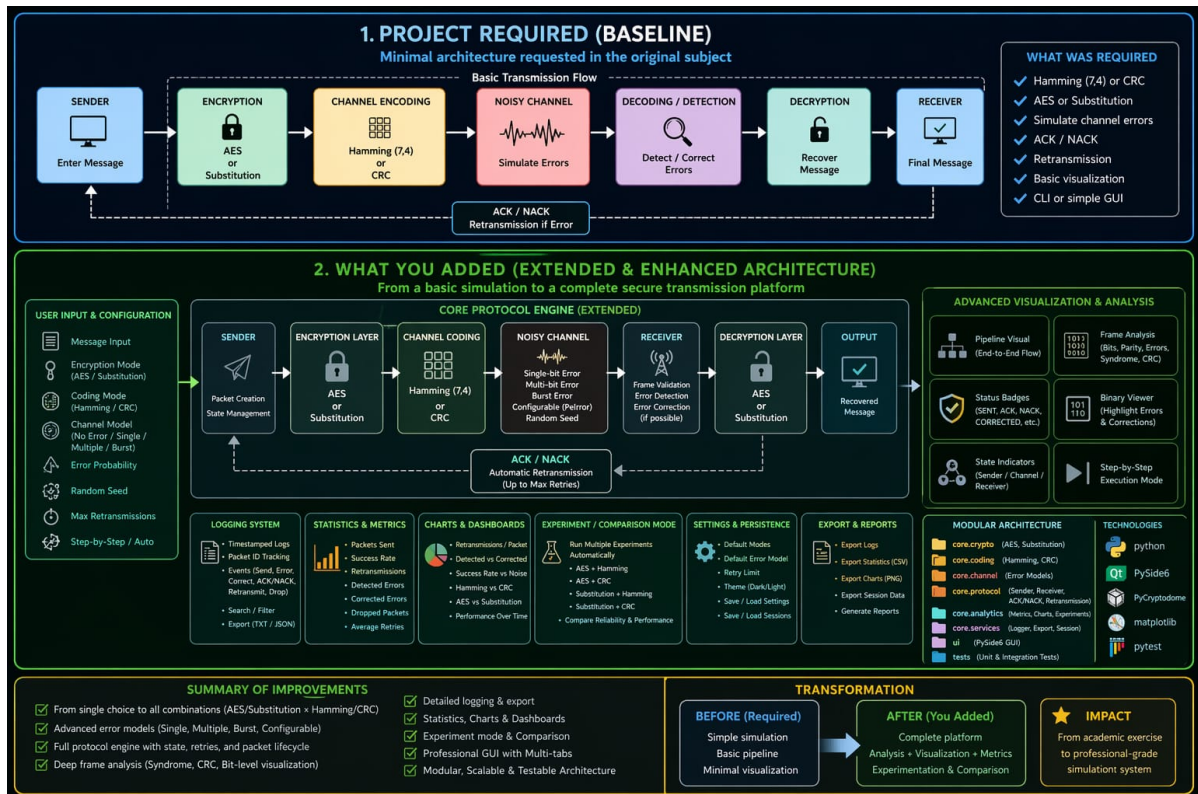


FIGURE 2.1 – Comparaison entre l'architecture minimale demandée par le sujet et l'architecture étendue réalisée.

Le schéma met en évidence deux niveaux :

- l'architecture de base imposée par le sujet ;
- l'architecture enrichie apportée dans la version finale, avec visualisation, analytics, exports et paramétrage persistant.

2.2 Schema logique de haut niveau

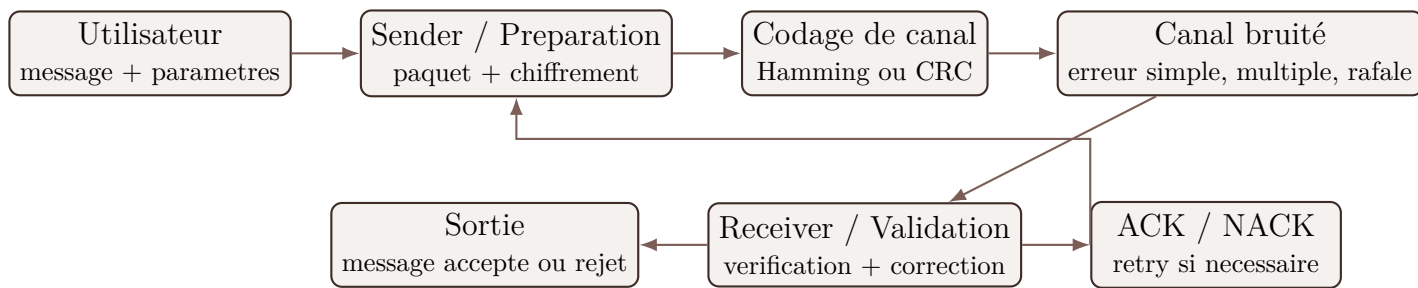


FIGURE 2.2 – Organisation logique du simulateur et boucle de retransmission.

Ce schema resume la philosophie du projet : une chaine de traitement principale enrichie par une boucle de retour protocolaire. L'information circule d'abord de gauche a droite jusqu'a la reception, puis un mecanisme de decision determine si la transmission est acceptee ou relancee. Cette separation claire entre flux principal et boucle de controle rend l'architecture plus facile a expliquer et a maintenir.

2.3 Principes de conception

Les principaux principes retenus sont :

- **separation des responsabilites** : chaque package a un role cible ;
- **testabilite** : les composants critiques sont independants de la GUI ;
- **observabilite** : l'application expose l'etat du protocole, les logs et les metriques ;
- **extensibilite** : de nouveaux modes d'erreur, codecs ou vues UI peuvent etre ajoutes sans casser le coeur du systeme.

Le projet repose egalement sur une vision orientee demonstration. Autrement dit, les objets de resultats, les etats intermediaires et les logs ne servent pas uniquement a l'execution interne ; ils servent aussi a l'explication. Cette orientation a influence plusieurs choix de conception, notamment la conservation des etapes de transformation du message et la production de structures de donnees directement exploitables par les onglets de l'interface.

3. Structure du projet dans VS Code

Le depot a ete organise sous forme de packages Python afin de refleter les couches fonctionnelles de l'application. La structure visible dans VS Code est la suivante.

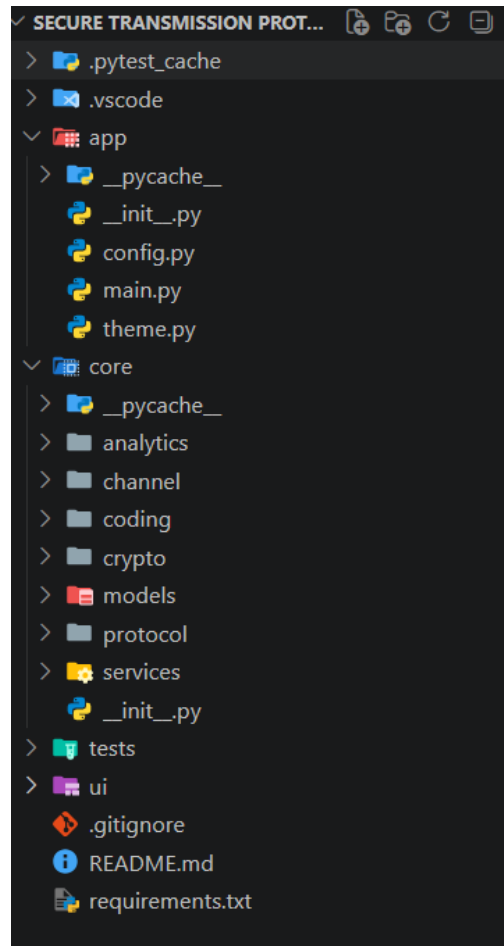


FIGURE 3.1 – Arborescence principale du projet dans VS Code.

3.1 Lecture de l'arborescence

Package / dossier	Role principal
app	Point d'entree de l'application, configuration persistante et theme visuel.
core/models	Modeles de donnees, dataclasses et etats de transmission.
core/crypto	Chiffrement AES, chiffrement par substitution et fonctions utilitaires.
core/coding	Manipulation binaire, Hamming(7,4) et CRC.
core/channel	Modeles d'erreurs et canal bruite parametrable.
core/protocol	Sender, receiver, reponses ACK/NACK, retransmission et controleur central.

Package / dossier	Rôle principal
core/analytics	Calcul des metriques, graphiques matplotlib et experiment runner.
core/services	Journalisation, export et sauvegarde de session.
ui	Fenetre principale, onglets, widgets reutilisables et dialogues.
tests	Tests unitaires et tests d'integration.

Cette arborescence montre une separation nette entre les couches metier et la couche de presentation. Les modules cryptographiques, le codage de canal, le protocole et l'analytics peuvent etre testes sans la GUI. A l'inverse, l'interface se contente principalement d'orchestrer l'affichage et l'interaction utilisateur autour d'objets deja prepares par le coeur.

4. Description detaillee des modules de code

4.1 Package **app**

Le package `app` contient `main.py`, `config.py` et `theme.py`. `main.py` initialise `QApplication`, charge les parametres par default et ouvre la fenetre principale. `config.py` centralise les reglages persistants du simulateur : mode de chiffrement, mode de codage, modele d'erreur, probabilite d'erreur et nombre maximal de retransmissions. `theme.py` definit la charte visuelle sombre de l'application.

4.2 Package **core.crypto**

Cette couche implante les mecanismes de confidentialite :

- `aes_cipher.py` fournit un chiffrement AES en mode CBC avec IV aleatoire et padding PKCS#7;
- `substitution_cipher.py` realise un chiffrement pedagogique simple pour la comparaison ;
- `crypto_utils.py` gere les conversions utiles, notamment entre bytes, texte et representations transportables.

Le choix d'avoir deux methodes de chiffrement est pedagogiquement pertinent : AES represente une solution serieuse de cryptographie symetrique, tandis que la substitution sert de point de comparaison simple pour observer les differences de comportement dans le simulateur.

Dans le fonctionnement interne, la couche de chiffrement travaille sur le message clair et retourne une representation transportable qui pourra ensuite etre convertie en bits pour le codage de canal. Cette separation entre « confidentialite » et « protection contre les erreurs » correspond a la logique theorique du module.

4.3 Package **core.coding**

Ce package regroupe toute la logique de protection contre les erreurs :

- `bit_utils.py` convertit les messages en chaines binaires et inversement ;
- `hamming74.py` implemente l'encodage, le syndrome, la detection et la correction simple ;
- `crc_codec.py` genere la trame CRC et verifie le reste a la reception.

Le code Hamming (7,4) est utile pour illustrer la correction d'une erreur simple. Le CRC, lui, sert surtout a la detection d'erreurs. L'application permet ainsi de comparer une logique de correction locale avec une logique de validation menant plus souvent a une retransmission.

Cette difference est fondamentale dans la lecture des resultats : avec Hamming, certaines trames peuvent etre recuperees localement sans repasser par l'emetteur ; avec CRC, la stra-

tegie est plus stricte et s'appuie davantage sur le protocole de retour pour obtenir une trame saine.

4.4 Package **core.channel**

La couche canal est responsable de la degradation volontaire de la trame. `error_models.py` enumere les differents types d'erreurs disponibles : aucune erreur, erreur aleatoire simple, erreurs multiples et erreur en rafale. `noisy_channel.py` applique ensuite concretement les inversions de bits selon une probabilite et une seed eventuelle. La reproductibilite apporte un vrai plus lors de la demonstration.

4.5 Package **core.protocol**

Il s'agit du coeur fonctionnel du projet.

- `sender.py` prepare le paquet, chiffre le message puis l'encode avant emission ;
- `receiver.py` valide, corrige si possible, puis decrypte ;
- `ack_nack.py` modele les reponses de retour ;
- `retransmission.py` decide si une nouvelle tentative doit etre declenchee ;
- `transmission_controller.py` orchestre tout le cycle de vie d'une transmission.

Cette organisation evite de melanger la logique de protocole avec l'interface. Le controleur retourne des resultats structures que la GUI peut afficher sans reimplementer la logique metier.

Le `transmission_controller` est la piece centrale car il coordonne les tentatives, collecte les informations produites a chaque etape et synthetise l'etat final de la session. Il alimente directement les vues de simulation, d'analyse, de logs et de statistiques.

4.6 Packages **core.analytics** et **core.services**

Le package `analytics` calcule les indicateurs de fiabilite et alimente les graphiques. Le package `services` prend en charge la journalisation, les exports texte / JSON / CSV et la sauvegarde de session. Ces couches rendent l'application exploitable en contexte de soutenance, de demonstration ou de comparaison experimentale.

4.7 Package **ui**

L'interface graphique repose sur PySide6 et une fenetre principale a six onglets :

- Simulation ;
- Frame Analysis ;
- Logs ;
- Statistics ;

- Comparison ;
- Settings.

Des widgets specialises sont reutilises pour le pipeline visuel, l'inspecteur de trame, les badges d'etat et les cartes de metriques. Cette approche limite la duplication de code et garantit une coherence visuelle sur l'ensemble de l'application.

4.8 Schema des dependances fonctionnelles

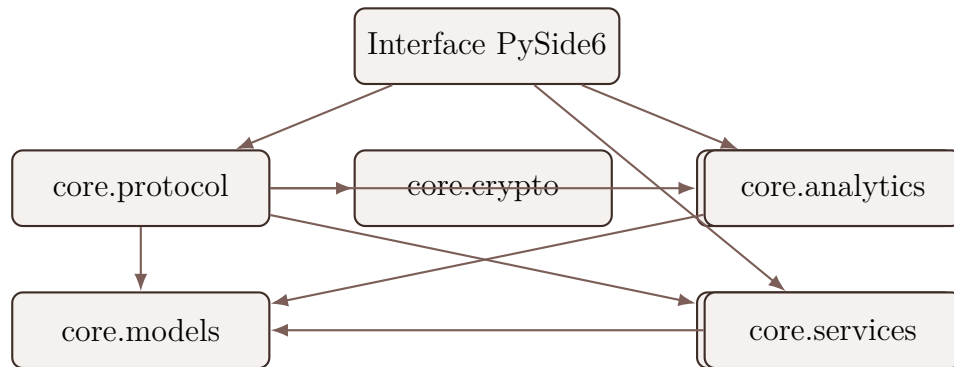


FIGURE 4.1 – Relations fonctionnelles principales entre les packages Python.

Ce schema montre que l'interface ne depend pas directement des details bas niveau de chaque algorithme. Elle dialogue avant tout avec les couches d'orchestration, d'analyse et de service. Cette approche favorise la reusabilite du code metier et reduit les effets de bord.

5. Flux fonctionnel du protocole

Le fonctionnement general du simulateur peut etre decrit en une suite d'etapes ordonnees :

1. l'utilisateur saisit un message et choisit ses parametres ;
2. le sender chiffre le message ;
3. la charge utile chiffee est encodee via Hamming ou CRC ;
4. la trame traverse un canal potentiellement bruite ;
5. le receiver valide ou corrige la trame recue ;
6. un ACK ou un NACK est emis ;
7. en cas d'echec, une retransmission est tentee jusqu'au maximum autorise ;
8. les logs et les statistiques sont mis a jour apres chaque etape significative.

5.1 Schema sequentiel de la transmission

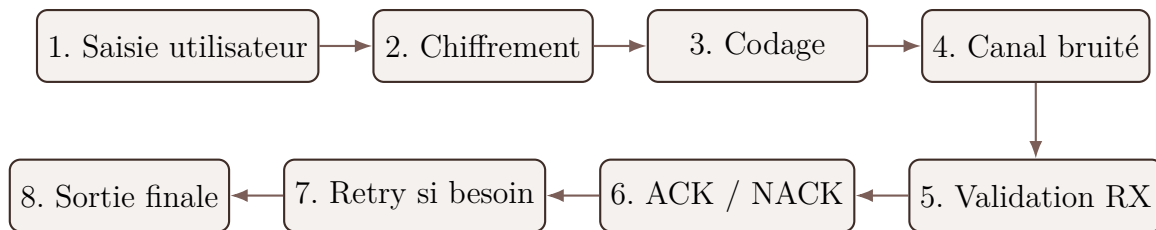


FIGURE 5.1 – Sequence generale d'une transmission simulee.

Le point important est que chaque etape produit une information visible dans l'application. Le message initial, la charge utile chiffee, la trame codee, la trame recue et le message de sortie sont tous exposes dans l'onglet Simulation. Cette transparence est un vrai atout pour comprendre le protocole, car elle relie directement theorie et observation.

5.2 Interet pedagogique de ce flux

Ce pipeline rend visible l'enchainement entre securite et fiabilite :

- le chiffrement protege le contenu ;
- le codage et le controle d'integrite traitent la corruption binaire ;
- le protocole de retour gere la robustesse face a l'echec ;
- la GUI expose ces transformations de maniere comprehensible.

Lorsque la transmission reussit, l'utilisateur voit le parcours complet jusqu'au message final. Lorsque la transmission echoue, l'application reste tout aussi instructive : elle montre preci-

sement a quel moment la trame devient invalide, pourquoi un NACK est envoye et comment le nombre maximal de tentatives conduit eventuellement a l'abandon du paquet.

6. Analyse de l'interface graphique

6.1 Onglet Simulation

L'onglet Simulation joue le role de salle de controle principale. Il concentre :

- les parametres d'entree du scenario ;
- les badges d'etat du protocole ;
- le pipeline emetteur-canal-recepteur ;
- les panneaux successifs du plaintext jusqu'au message final.

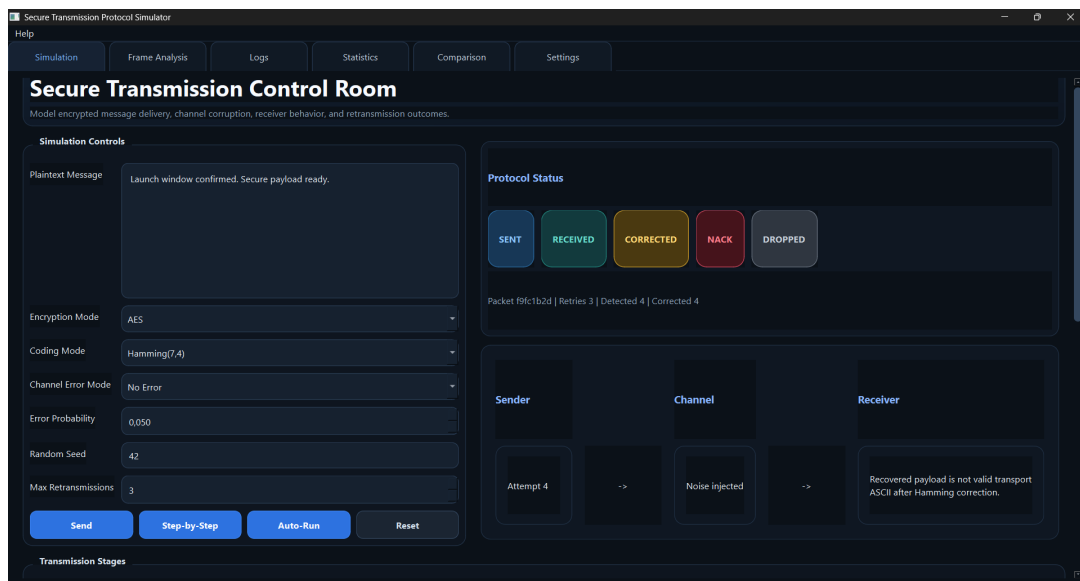


FIGURE 6.1 – Vue generale de l'onglet Simulation : controles, badges, statut du protocole et pipeline.

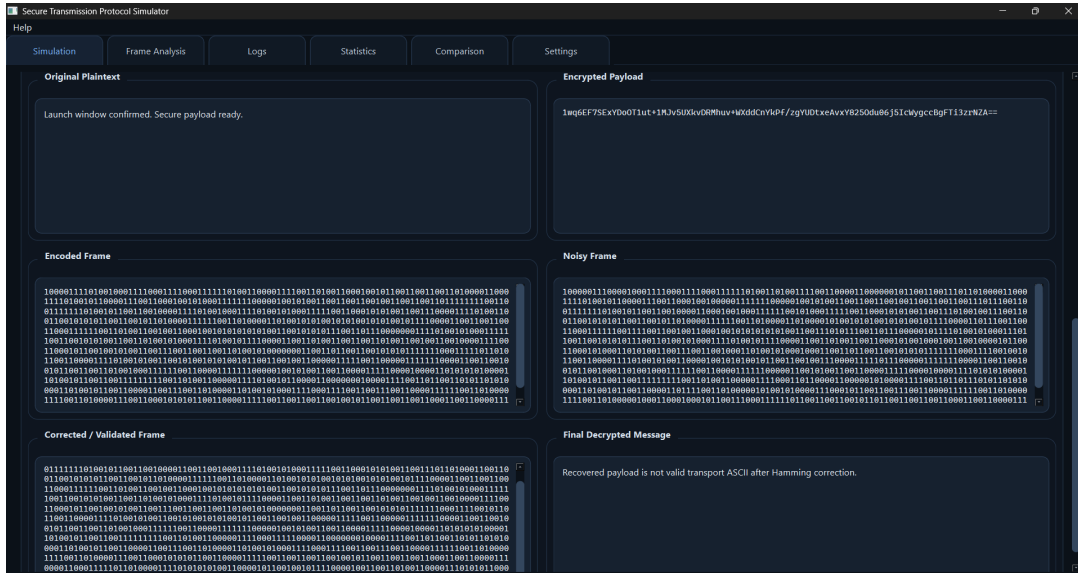


FIGURE 6.2 – Détails des etapes de transmission : texte initial, payload chiffre, frame codee, frame bruitée et resultat final.

Cette vue donne une lecture immédiatement compréhensible du comportement du système. Le côté gauche regroupe les entrées utilisateur, tandis que le côté droit suit l'état de la transmission. Les panneaux inférieurs permettent d'observer l'effet exact des traitements.

Du point de vue ergonomique, cet onglet est le cœur de la démonstration. Il permet de passer rapidement d'un scénario propre à un scénario dégradé simplement en changeant le mode d'erreur, la probabilité ou la seed. Les badges d'état rendent le résultat lisible en quelques secondes, tandis que les panneaux de détail apportent le niveau d'explication nécessaire à une maintenance technique.

6.2 Onglet Frame Analysis

L'onglet « Frame Analysis » constitue un inspecteur binaire. Il met en évidence :

- les positions de parité ;
- les bits modifiés par le canal ;
- les bits corrigés à la réception ;
- le syndrome Hamming ou le reste CRC ;
- les métadonnées de la trame.

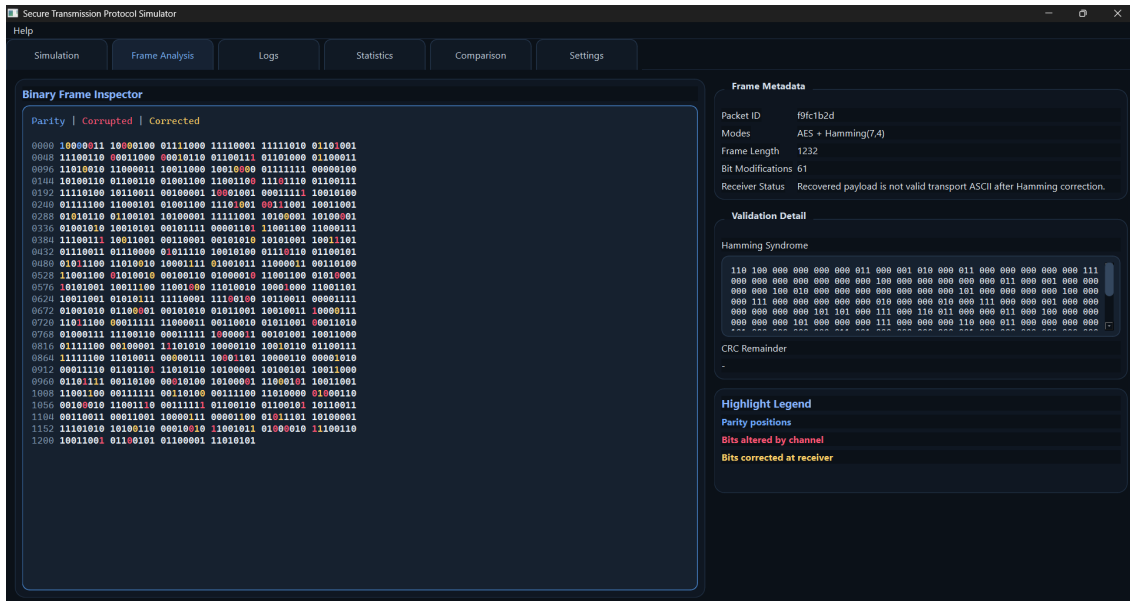


FIGURE 6.3 – Inspection détaillée de la trame binaire et metadonnées de validation.

Cette vue est particulièrement utile pour expliquer les algorithmes de détection / correction de façon concrète lors d’une démonstration.

Elle sert également à distinguer les deux approches de protection de l’information. Avec Hamming, l’accent est mis sur la localisation et la correction d’un bit fautif. Avec CRC, l’accent porte davantage sur la vérification d’intégrité et la décision de redemander ou non une nouvelle émission.

6.3 Onglet Logs

Le journal des événements fournit une trace chronologique horodatée des opérations du protocole. Il affiche notamment la préparation du paquet, l’envoi, le passage dans le canal, les NACK, les retries et l’éventuel abandon après épuisement des tentatives.

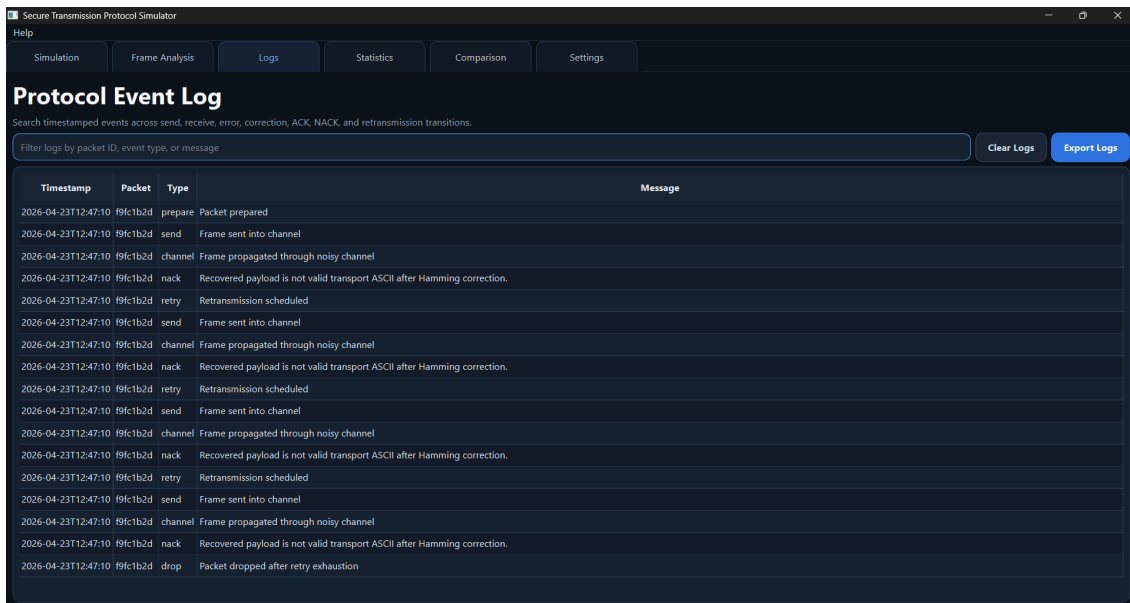


FIGURE 6.4 – Journal des evenements du protocole avec filtrage et export.

Cet onglet transforme le simulateur en outil d'observation. Il ne se contente pas d'afficher des messages ; il structure la chronologie du protocole. L'ordre des lignes permet de suivre la vie du paquet depuis sa preparation jusqu'a sa reussite ou son abandon, ce qui facilite fortement l'analyse d'un scenario problematique.

6.4 Onglet Statistics

Le tableau de bord statistique presente les indicateurs consolides de la session : nombre total de paquets, succes, retransmissions, erreurs detectees, corrections, taux de succes et nombre moyen de retries. Il integre egalement des graphiques matplotlib pour la lecture rapide des tendances.

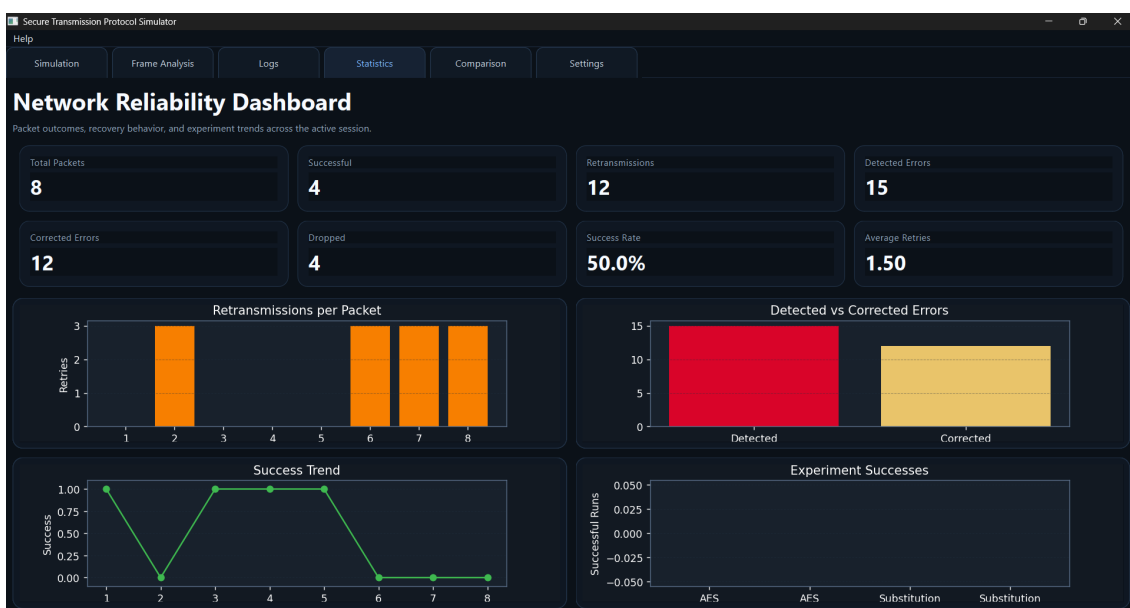


FIGURE 6.5 – Tableau de bord statistique et graphiques de suivi des transmissions.

L'interet de cette vue est de prendre de la distance par rapport a une transmission unique. Au lieu d'analyser seulement un cas, l'utilisateur peut observer des tendances : hausse du nombre de retries, chute du taux de succes, ecart entre erreurs detectees et erreurs corrigees, ou encore comportement plus ou moins robuste selon le bruit injecte.

6.5 Onglet Comparison

Le mode « Comparison » automatise des campagnes de tests sur les combinaisons : AES + Hamming, AES + CRC, Substitution + Hamming et Substitution + CRC. Il produit une matrice de resultats permettant de comparer fiabilite, retransmissions, corrections et paquets rejetes.

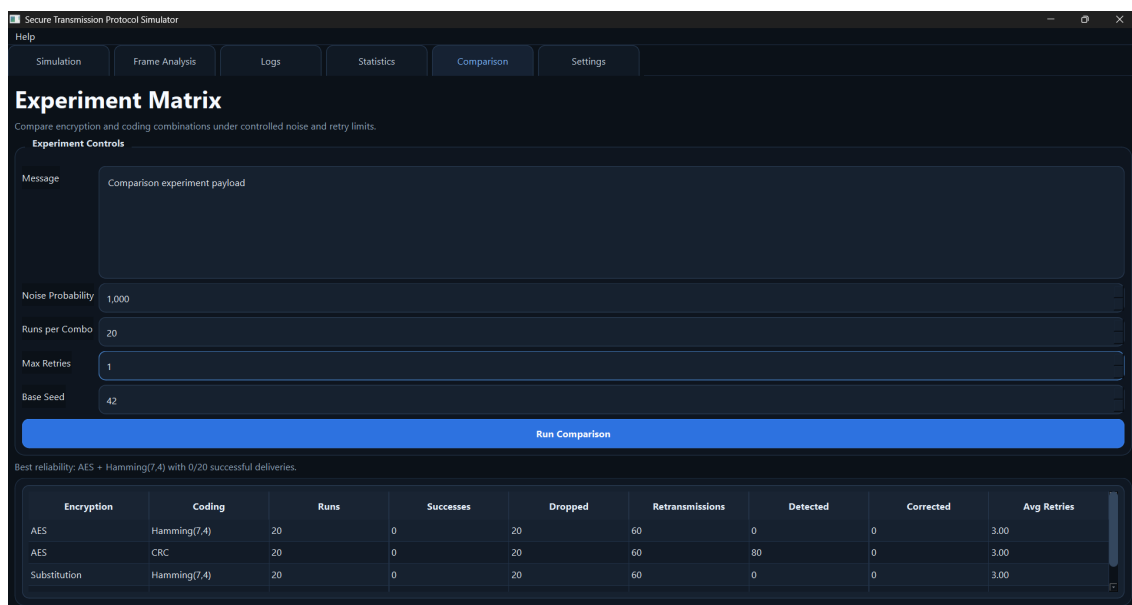


FIGURE 6.6 – Mode de comparaison experimentale entre configurations de chiffrement et de codage.

Le mode Comparison est particulierement utile pour repondre a une question de type « quelle combinaison se comporte le mieux dans un contexte donne? ». Il fait donc le lien entre demonstration fonctionnelle et petite experimentation encadree.

6.6 Onglet Settings

L'onglet Settings centralise les valeurs par default du simulateur afin de stabiliser les demonstrations et d'assurer une reutilisation simple de l'outil.

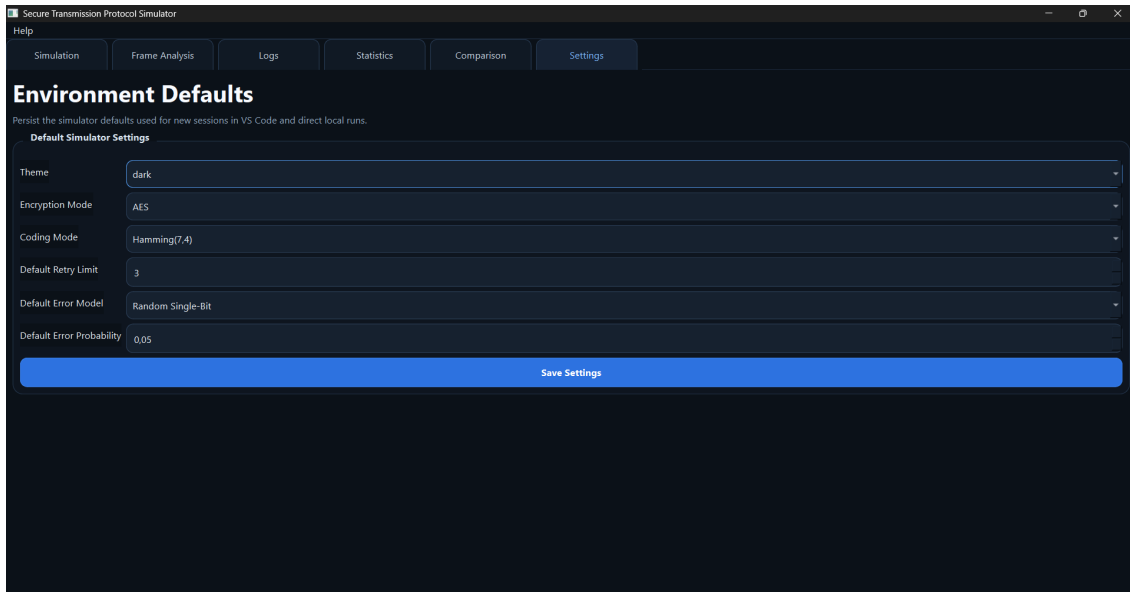


FIGURE 6.7 – Parametrage des options par défaut du simulateur.

Cet onglet contribue à la reproductibilité des essais. En imposant des valeurs par défaut, l'utilisateur peut retrouver rapidement les mêmes conditions de démonstration sans ressaisir l'ensemble des paramètres à chaque lancement.

7. Validation, qualite logicielle et tests

7.1 Strategie de validation

La qualite du projet ne repose pas uniquement sur l'observation visuelle de la GUI. Des tests automatises ont ete ecrits pour les composants critiques :

- chiffrement / dechiffrement AES ;
- round trip du chiffrement par substitution ;
- encodage, decodage et correction Hamming(7,4) ;
- validation CRC ;
- injection d'erreurs dans le canal ;
- logique ACK/NACK et retransmission ;
- scenario d'integration de bout en bout.

Cette strategie de validation est coherente avec l'architecture modulaire du projet. Comme les composants sont decouples, chacun peut etre verifie a son niveau avant d'etre reintegre dans le simulateur global. Cela renforce la confiance dans les resultats observes dans l'interface.

7.2 Apports de cette batterie de tests

Les tests assurent plusieurs garanties :

- la logique metier reste correcte meme sans l'interface ;
- les regressions sont detectees plus vite lors des evolutions ;
- les couches critiques restent modulaires et verifiables ;
- la soutenance peut s'appuyer sur des preuves concretes de fiabilite logicielle.

7.3 Lien entre tests et experience utilisateur

Un point important de ce projet est que la qualite technique a un effet direct sur l'experience utilisateur. Si l'encodage, le canal ou le protocole etaient fragiles, l'interface afficherait des etats incoherents ou des resultats trompeurs. Le travail de test ne releve donc pas seulement de la qualite interne du code ; il participe a la credibilite pedagogique du simulateur.

8. Analyse critique et apports du projet

8.1 Apports principaux

Par rapport au sujet minimal, plusieurs enrichissements ont été apportés :

- application desktop complète et non simple script de démonstration ;
- visualisation multi-niveaux du message et de la trame ;
- mode comparaison pour analyser les stratégies ;
- logs exportables et statistiques exploitables ;
- architecture modulaire propre et testable.

L'un des apports majeurs est aussi la transformation du sujet initial en objet démonstratif. Le projet n'est pas uniquement correct ; il est explicable. Cette qualité est essentielle dans un cadre universitaire, car elle permet de justifier les choix techniques, de montrer la maîtrise des concepts et d'illustrer clairement la relation entre théorie et implémentation.

8.2 Limites

Malgré ses qualités, le projet conserve des limites normales dans un cadre pédagogique :

- le protocole reste volontairement simple ;
- Hamming(7,4) ne corrige pas les erreurs multiples importantes ;
- la simulation ne reproduit pas l'ensemble des subtilités d'un protocole réseau réel ;
- certains choix ont été privilégiés pour la clarté didactique plutôt que pour une implémentation industrielle exhaustive.

8.3 Pistes d'évolution

Des améliorations futures peuvent être envisagées :

- ajout d'autres codes correcteurs ;
- prise en charge de tailles de trame variables plus complexes ;
- animation temps réel plus fine du pipeline ;
- export automatisé de rapports d'expérience ;
- comparaison de performances temporelles plus détaillée.

Conclusion

Le projet « Secure Transmission Protocol Simulator » répond au besoin académique du module « Transmission de données sécurisée » en proposant une réalisation à la fois correcte sur le plan algorithmique, propre sur le plan logiciel et convaincante sur le plan visuel. Il illustre de manière concrète la chaîne complète de transmission : préparation du message, chiffrement, codage, perturbation, validation, correction éventuelle, ACK/NACK et retransmission.

Au-delà de la fonctionnalité attendue, l'application fournit un environnement de démonstration riche : interface claire, visualisation des bits, tableau de bord, comparaison expérimentale, exports et tests automatisés. Elle constitue ainsi un support de soutenance sérieux et un bon point de départ pour des évolutions futures.

A. Annexe A - Synthèse des fonctionnalités

Cette annexe regroupe, sous forme compacte, les fonctions majeures disponibles dans l'application et leur utilité dans le cadre du module.

Fonctionnalité	Description et utilité
Chiffrement AES	Protection sérieuse du contenu du message avant transmission.
Chiffrement par substitution	Mode simple et pédagogique utilisé pour la comparaison expérimentale.
Codage Hamming(7,4)	Détection et correction d'une erreur simple, avec syndrome visible dans l'interface.
CRC	Détection d'intégrité de la trame et déclenchement éventuel d'une retransmission.
Canal bruité configurable	Simulation d'erreurs simples, multiples ou en rafale avec probabilité réglable.
ACK / NACK	Validation ou rejet de la transmission selon l'état de la trame reçue.
Retransmission automatique	Nouvelle tentative tant que le nombre maximal d'essais n'est pas atteint.
Analyse binaire	Visualisation des bits de parité, des bits altérés et des corrections appliquées.
Logs exportables	Trace exploitable en maintenance ou pour une analyse ultérieure.
Statistiques et graphiques	Lecture synthétique du comportement du simulateur sur plusieurs essais.
Mode Comparison	Mise en parallèle de plusieurs configurations pour juger leur robustesse relative.

B. Annexe B - Fichiers de code les plus importants

Les fichiers suivants constituent l'ossature principale du projet et peuvent être cités ou insérés en extrait dans une version enrichie du rapport :

- `app/main.py` : point d'entrée de l'application ;
- `app/config.py` : gestion des paramètres par défaut ;
- `core/crypto/aes_cipher.py` : chiffrement AES ;
- `core/crypto/substitution_cipher.py` : chiffrement pédagogique par substitution ;
- `core/coding/hamming74.py` : logique Hamming(7,4) ;
- `core/coding/crc_codec.py` : validation CRC ;
- `core/channel/noisy_channel.py` : simulation du canal bruité ;
- `core/protocol/transmission_controller.py` : orchestration complète du protocole ;
- `ui/main_window.py` : fenêtre principale et liaison des onglets ;
- `ui/simulation_tab.py` : tableau de bord de simulation.

Bibliographie

Bibliographie

- [1] Cyril Bloch, Aix-Marseille Universite, *Conseils pratiques pour la redaction des memoires*, guide methodologique consulte en ligne.
- [2] Universite de Lorraine, *Guide de redaction d'un rapport, memoire ou these*, document universitaire consulte en ligne.
- [3] Synthese des recommandations communes observees dans les guides universitaires : page de garde sobre, sommaire ou table des matieres, introduction, developpement structure, conclusion, bibliographie, annexes et pagination coherente.